

Machine Learning Lab (ISWE410P) - L51+L52

Lab Assessment - 2

Name : Sivashree.PL

Reg no : 23MIS0117

Date : 16/07/2026

Problem Statement 1 :

Dataset 1: Student Details

Student_ID	Name	Age	Gender	Department	CGPA
101	Alice	20	Female	CSE	8.5
102	Bob	NaN	Male	CSE	7.8
103	Charlie	21	Male	cse	9.1
104	David	-5	Male	ECE	8
105	Eva	22	Female	E.C.E	8.5
106	Frank	23	M	CSE	7.5
107	Grace	200	Female	CSE	8.2
108	Helen	24	Female	Computer Science	8.7
109	Ian	22	Female	EEE	NaN
110	Jack	21	Male	ECE	8.4

Dataset 2: Placement Details

Student_ID	Company	Package (LPA)
101	TCS	6.5
102	Infosys	5.8
103	Google	28
104	Accenture	7.2
105	Amazon	24
106	Wipro	5.5
107	Microsoft	32
108	Zoho	9
109	Cognizant	6
110	IBM	8

Task 1: Create both datasets as Pandas DataFrames.

[2]



```
from google.colab import files
uploaded = files.upload()
```



Choose Files 4 files

```
ratings.csv(text/csv) - 167 bytes, last modified: 7/16/2026 - 100% done
movies.csv(text/csv) - 438 bytes, last modified: 7/16/2026 - 100% done
placements.csv(text/csv) - 186 bytes, last modified: 7/16/2026 - 100% done
students.csv(text/csv) - 331 bytes, last modified: 7/16/2026 - 100% done
Saving ratings.csv to ratings.csv
Saving movies.csv to movies.csv
Saving placements.csv to placements.csv
Saving students.csv to students.csv
```



```
import pandas as pd
```

```
student = pd.read_csv("students.csv")
placement = pd.read_csv("placements.csv")
```

```
print("Student Dataset")
print(student.head())
```

```
print("\nPlacement Dataset")
print(placement.head())
```

... Student Dataset

	Student_ID	Name	Age	Gender	Department	CGPA
0	101	Alice	20.0	Female	CSE	8.5
1	102	Bob	NaN	Male	CSE	7.8
2	103	Charlie	21.0	Male	cse	9.1
3	104	David	-5.0	Male	ECE	8.0
4	105	Eva	22.0	Female	E.C.E	8.5

Placement Dataset

	Student_ID	Company	Package (LPA)
0	101	TCS	6.5
1	102	Infosys	5.8
2	103	Google	28.0
3	104	Accenture	7.2
4	105	Amazon	24.0

Task 2: Merge the two datasets using Student_ID.

```
merged = pd.merge(student, placement, on="Student_ID")
```

```
print(merged)
```

```
...   Student_ID  Name  Age  Gender  Department  CGPA  Company \
0         101  Alice  20.0  Female         CSE    8.5      TCS
1         102    Bob   NaN   Male         CSE    7.8  Infosys
2         103  Charlie  21.0   Male         cse    9.1   Google
3         104  David  -5.0   Male         ECE    8.0  Accenture
4         105    Eva  22.0  Female      E.C.E    8.5   Amazon
5         106  Frank  23.0      M         CSE    7.5    Wipro
6         107  Grace  200.0  Female         CSE    8.2  Microsoft
7         108  Helen  24.0  Female  Computer Science  8.7     Zoho
8         109    Ian  22.0  Female         EEE   NaN  Cognizant
9         110   Jack  21.0   Male         ECE    8.4     IBM
```

```
Package (LPA)
```

```
0         6.5
1         5.8
2        28.0
3         7.2
4        24.0
5         5.5
6        32.0
7         9.0
8         6.0
9         8.0
```

Task 3: Clean the dataset

```
# missing Age
merged["Age"] = merged["Age"].fillna(merged["Age"].mean())

# missing CGPA
merged["CGPA"] = merged["CGPA"].fillna(merged["CGPA"].mean())

# Correct Department names
merged["Department"] = merged["Department"].replace({
    "cse": "CSE",
    "Computer Science": "CSE",
    "E.C.E": "ECE"
})

# Correct Gender
merged["Gender"] = merged["Gender"].replace({
    "M": "Male"
})

# Remove invalid ages
merged = merged[(merged["Age"] >= 18) & (merged["Age"] <= 30)]

print(merged)
```

```
...      Student_ID  Name  Age  Gender  Department  CGPA  Company \
0           101    Alice  20.0  Female         CSE    8.5      TCS
2           103  Charlie  21.0   Male         CSE    9.1    Google
4           105      Eva  22.0  Female         ECE    8.5    Amazon
5           106    Frank  23.0   Male         CSE    7.5     Wipro
7           108    Helen  24.0  Female         CSE    8.7     Zoho
8           109      Ian  22.0  Female         EEE    8.3  Cognizant
9           110     Jack  21.0   Male         ECE    8.4     IBM

      Package (LPA)
0              6.5
2             28.0
4             24.0
5              5.5
7              9.0
8              6.0
9              8.0
```

Task 4: Display:

- Students with CGPA > 8.5

```
print(merged[merged["CGPA"]>8.5])
```

...	Student_ID	Name	Age	Gender	Department	CGPA	Company	Package (LPA)
2	103	Charlie	21.0	Male	CSE	9.1	Google	28.0
7	108	Helen	24.0	Female	CSE	8.7	Zoho	9.0

- Students placed in companies with Package > 10 LPA

```
print(merged[merged["Package (LPA)"]>10])
```

...	Student_ID	Name	Age	Gender	Department	CGPA	Company	Package (LPA)
2	103	Charlie	21.0	Male	CSE	9.1	Google	28.0
4	105	Eva	22.0	Female	ECE	8.5	Amazon	24.0

- Students belonging to the CSE department

```
print(merged[merged["Department"]=="CSE"])
```

...	Student_ID	Name	Age	Gender	Department	CGPA	Company	Package (LPA)
0	101	Alice	20.0	Female	CSE	8.5	TCS	6.5
2	103	Charlie	21.0	Male	CSE	9.1	Google	28.0
5	106	Frank	23.0	Male	CSE	7.5	Wipro	5.5
7	108	Helen	24.0	Female	CSE	8.7	Zoho	9.0

Task 5: Reshape the merged dataset to display only Student_ID, Name, Company, Package

Sort the output in descending order of Package.

```
result = merged[["Student_ID", "Name", "Company", "Package (LPA)"]]  
  
result = result.sort_values(by="Package (LPA)", ascending=False)  
  
print(result)
```

...	Student_ID	Name	Company	Package (LPA)
2	103	Charlie	Google	28.0
4	105	Eva	Amazon	24.0
7	108	Helen	Zoho	9.0
9	110	Jack	IBM	8.0
0	101	Alice	TCS	6.5
8	109	Ian	Cognizant	6.0
5	106	Frank	Wipro	5.5

Dataset 3: Movies.csv

Movie_ID	Title	Genre	Duration (min)	Language	Year
101	Inception	Sci-Fi	148	English	2010
102	Avatar	Action	162	English	2009
103	Interstellar	Sci-Fi	169	English	2014
104	Frozen	Animation	102	English	2013
105	Jawan	Action	170	Hindi	2023
106	Leo	Action	163	Tamil	2023
107	The Dark Knight	Action	152	English	2008
108	3 Idiots	Comedy	171	Hindi	2009
109	Kalki 2898 AD	Sci-Fi	181	Telugu	2024
110	Inside Out 2	Animation	96	English	2024

Dataset 4: Ratings.csv

Movie_ID	IMDb_Rating	Votes (Millions)
101	8.8	2.9
102	7.9	1.5
103	8.7	2.3
104	7.5	0.8
105	7.2	0.5
106	8.1	0.7
107	9	2.8
108	8.4	0.9
109	8.3	0.6
110	7.9	0.4

Problem Statement 2 :

Task 1: Load both CSV files into Pandas DataFrames.

Display:

- First 5 rows
- Shape of each dataset
- Dataset information (info())

```
import pandas as pd

movies = pd.read_csv("movies.csv")
ratings = pd.read_csv("ratings.csv")

print("Movies Dataset")
print(movies.head())

print("\nRatings Dataset")
print(ratings.head())

print("\nMovies Shape:", movies.shape)
print("Ratings Shape:", ratings.shape)

print("\nMovies Info")
movies.info()

print("\nRatings Info")
ratings.info()
```

... Movies Dataset

	Movie_ID	Title	Genre	Duration (min)	Language	Year
0	101	Inception	Sci-Fi	148	English	2010
1	102	Avatar	Action	162	English	2009
2	103	Interstellar	Sci-Fi	169	English	2014
3	104	Frozen	Animation	102	English	2013
4	105	Jawan	Action	170	Hindi	2023

Ratings Dataset

	Movie_ID	IMDb_Rating	Votes (Millions)
0	101	8.8	2.9
1	102	7.9	1.5
2	103	8.7	2.3
3	104	7.5	0.8
4	105	7.2	0.5

Movies Shape: (10, 6)

Ratings Shape: (10, 3)

Movies Info

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 10 entries, 0 to 9

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	Movie_ID	10 non-null	int64
1	Title	10 non-null	object
2	Genre	10 non-null	object
3	Duration (min)	10 non-null	int64
4	Language	10 non-null	object
5	Year	10 non-null	int64

dtypes: int64(3), object(3)

memory usage: 612.0+ bytes

Ratings Info

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 10 entries, 0 to 9

Data columns (total 3 columns):

#	Column	Non-Null Count	Dtype
0	Movie_ID	10 non-null	int64
1	IMDb_Rating	10 non-null	float64
2	Votes (Millions)	10 non-null	float64

dtypes: float64(2), int64(1)

memory usage: 372.0 bytes

Task 2: Merge the two datasets using Movie_ID.

Display the merged dataset.

```
merged = pd.merge(movies, ratings, on="Movie_ID")

print(merged)
```

...	Movie_ID	Title	Genre	Duration (min)	Language	Year	\
0	101	Inception	Sci-Fi	148	English	2010	
1	102	Avatar	Action	162	English	2009	
2	103	Interstellar	Sci-Fi	169	English	2014	
3	104	Frozen	Animation	102	English	2013	
4	105	Jawan	Action	170	Hindi	2023	
5	106	Leo	Action	163	Tamil	2023	
6	107	The Dark Knight	Action	152	English	2008	
7	108	3 Idiots	Comedy	171	Hindi	2009	
8	109	Kalki 2898 AD	Sci-Fi	181	Telugu	2024	
9	110	Inside Out 2	Animation	96	English	2024	

	IMDb_Rating	Votes (Millions)
0	8.8	2.9
1	7.9	1.5
2	8.7	2.3
3	7.5	0.8
4	7.2	0.5
5	8.1	0.7
6	9.0	2.8
7	8.4	0.9
8	8.3	0.6
9	7.9	0.4

Task 3: Preprocess the dataset

```
# Check missing values
print(merged.isnull().sum())

# Fill missing values
merged = merged.fillna(method="ffill")

print("\nAfter Cleaning")
print(merged)
```

```
... Movie_ID      0
    Title         0
    Genre         0
    Duration (min) 0
    Language      0
    Year          0
    IMDb_Rating   0
    Votes (Millions) 0
    dtype: int64
```

After Cleaning

	Movie_ID	Title	Genre	Duration (min)	Language	Year	\
0	101	Inception	Sci-Fi	148	English	2010	
1	102	Avatar	Action	162	English	2009	
2	103	Interstellar	Sci-Fi	169	English	2014	
3	104	Frozen	Animation	102	English	2013	
4	105	Jawan	Action	170	Hindi	2023	
5	106	Leo	Action	163	Tamil	2023	
6	107	The Dark Knight	Action	152	English	2008	
7	108	3 Idiots	Comedy	171	Hindi	2009	
8	109	Kalki 2898 AD	Sci-Fi	181	Telugu	2024	
9	110	Inside Out 2	Animation	96	English	2024	

	IMDb_Rating	Votes (Millions)
0	8.8	2.9
1	7.9	1.5
2	8.7	2.3
3	7.5	0.8
4	7.2	0.5
5	8.1	0.7
6	9.0	2.8
7	8.4	0.9
8	8.3	0.6
9	7.9	0.4

Task 4: Filter the merged dataset to display:

1. Movies with IMDb Rating greater than 8.5

2024

```
▶ print(merged[merged["IMDb_Rating"] > 8.5])
```

```
...   Movie_ID      Title  Genre  Duration (min)  Language  Year  \
0      101      Inception  Sci-Fi          148    English  2010
2      103  Interstellar  Sci-Fi          169    English  2014
6      107  The Dark Knight  Action          152    English  2008

   IMDb_Rating  Votes (Millions)
0           8.8              2.9
2           8.7              2.3
6           9.0              2.8
```

2. Movies released after 2020

```
print(merged[merged["Year"] > 2020])
```

...	Movie_ID	Title	Genre	Duration (min)	Language	Year	\
4	105	Jawan	Action	170	Hindi	2023	
5	106	Leo	Action	163	Tamil	2023	
8	109	Kalki 2898 AD	Sci-Fi	181	Telugu	2024	
9	110	Inside Out 2	Animation	96	English	2024	

	IMDb_Rating	Votes (Millions)
4	7.2	0.5
5	8.1	0.7
8	8.3	0.6
9	7.9	0.4

3. Only Sci-Fi movies

```
print(merged[merged["Genre"] == "Sci-Fi"])
```

...	Movie_ID	Title	Genre	Duration (min)	Language	Year	\
0	101	Inception	Sci-Fi	148	English	2010	
2	103	Interstellar	Sci-Fi	169	English	2014	
8	109	Kalki 2898 AD	Sci-Fi	181	Telugu	2024	

	IMDb_Rating	Votes (Millions)
0	8.8	2.9
2	8.7	2.3
8	8.3	0.6

Task 5: Reshape the dataset to display only the following columns: Title, Genre, IMDb_Rating, Duration, Language

Sort the output in descending order of IMDb Rating.

Create one suitable visualization (Bar Chart, Pie Chart, or Histogram) using Matplotlib.

Add appropriate chart title and axis labels.

Finally, save the cleaned dataset as: netflix_cleaned.csv

```

▶ result = merged[["Title", "Genre", "IMDb_Rating", "Duration (min)", "Language"]]

result = result.sort_values(by="IMDb_Rating", ascending=False)

print(result)

```

```

...
      Title      Genre  IMDb_Rating  Duration (min)  Language
6  The Dark Knight   Action         9.0           152    English
0      Inception   Sci-Fi         8.8           148    English
2    Interstellar   Sci-Fi         8.7           169    English
7      3 Idiots    Comedy         8.4           171     Hindi
8  Kalki 2898 AD   Sci-Fi         8.3           181    Telugu
5          Leo     Action         8.1           163     Tamil
1      Avatar     Action         7.9           162    English
9  Inside Out 2  Animation         7.9            96    English
3      Frozen     Animation         7.5           102    English
4      Jawan     Action         7.2           170     Hindi

```

```

▶ import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))

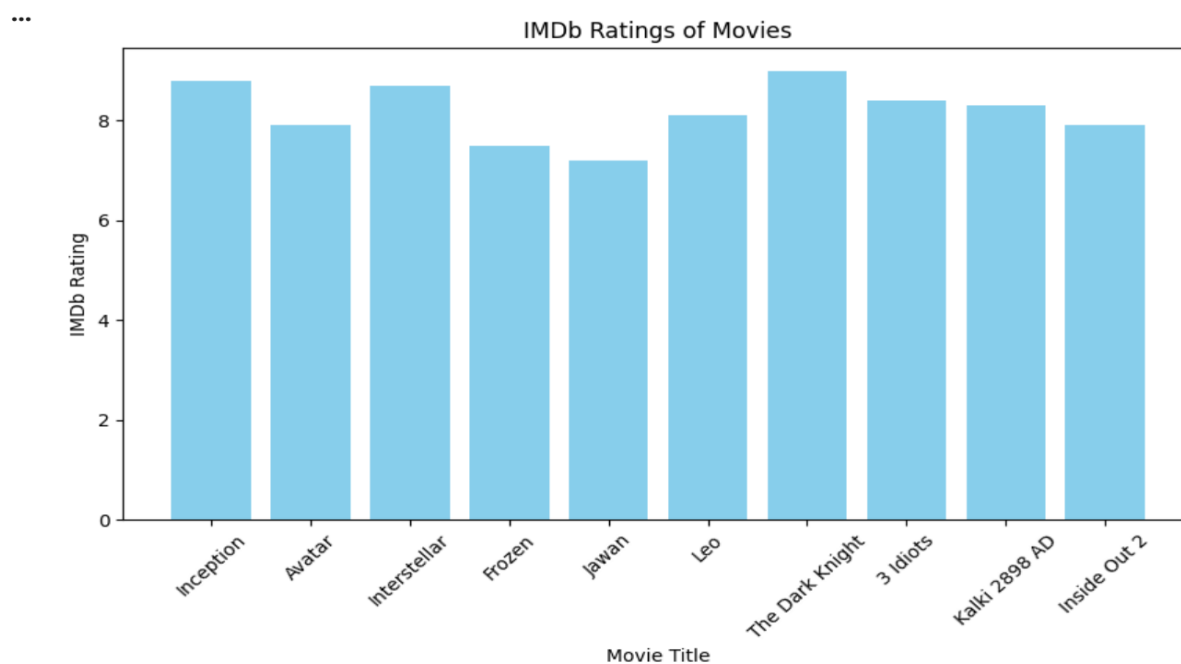
plt.bar(merged["Title"], merged["IMDb_Rating"], color="skyblue")

plt.title("IMDb Ratings of Movies")
plt.xlabel("Movie Title")
plt.ylabel("IMDb Rating")

plt.xticks(rotation=45)

plt.show()

```



```
merged.to_csv("netflix_cleaned.csv", index=False)

print("netflix_cleaned.csv saved successfully!")

... netflix_cleaned.csv saved successfully!
```

Answer the following:

1. Why is data preprocessing considered an important step before building a Machine Learning model?

Data preprocessing improves the quality of data. It removes errors, missing values, and duplicate data. Clean data helps the Machine Learning model give better and more accurate results.

2. When would you choose fillna() over dropna() for handling missing values? Explain your reasoning.

Use fillna() when you do not want to lose data. It replaces missing values with the mean, median, or another value. dropna() removes rows with missing values.

3. What is the purpose of merging two datasets? What could happen if the wrong key is used for merging?

Merging combines related data from two datasets into one. If the wrong key is used, the data may not match correctly, leading to incorrect or missing information.

4. Differentiate between filtering and reshaping a dataset with one example for each.

Filtering selects rows based on a condition.

- Example:

```
student[student["CGPA"] > 8.5]
```

Reshaping selects only required columns.

- Example:

```
student[["Name","CGPA"]]
```

5. After cleaning the dataset, how would you verify that no missing values remain in the DataFrame?

```
print(merged.isnull().sum())
```

```
... Movie_ID      0
    Title         0
    Genre         0
    Duration (min)  0
    Language      0
    Year          0
    IMDb_Rating   0
    Votes (Millions)  0
    dtype: int64
```